

Projet de Jeu Vintage en Java

Architecture MVC et Design Patterns

Membre 1: Djenaba BARRY- Graphes & Pathfinding

Membre 2: Ayman EL GHALI- Entités & Factory

Membre 3: Souhila BENACHOUR- MVC & Interface

Université de Bordeaux

Master 4TIN706U - Approche Objet

Décembre 2025

Table des matières

1	Introduction	4
1.1	Objectifs du Projet	4
1.2	Scénario du Jeu	4
2	Répartition des Responsabilités	4
2.1	Membre 1 - Graphes et Pathfinding	4
2.2	Membre 2 - Entités et Factory Pattern	4
2.3	Membre 3 - MVC et Interface	5
3	Architecture Globale : Le Modèle MVC	5
3.1	Le Modèle (<code>fr.ubordeaux.ao.project.model</code>)	5
3.2	La Vue (<code>fr.ubordeaux.ao.project.view</code>)	5
3.3	Le Contrôleur (<code>fr.ubordeaux.ao.project.controller</code>)	6
4	Design Patterns Implémentés	6
4.1	Pattern Observer (Comportement)	6
4.2	Pattern Factory (Création)	6
4.3	Pattern Singleton (Configuration)	7
4.4	Pattern MVC (Architectural)	7
5	Algorithmique et Théorie des Graphes	7
5.1	Modélisation du Labyrinthe	7
5.2	Algorithme de Recherche de Chemin (Pathfinding)	8
6	Gestion des Entités du Jeu	9
6.1	La Classe Enemy	9
6.2	Gestion des Clés, Portes et Sorties	9
7	Qualité du Code (Clean Code)	10
7.1	Nommage Significatif	10
7.2	Méthodes Courtes	10
7.3	Gestion des Exceptions	10
7.4	Documentation	10
8	Structure du Projet	11
8.1	Organisation des Packages	11
8.2	Ressources	11
9	Fonctionnalités du Jeu	12
9.1	Niveaux	12
9.2	Gameplay	12
10	Utilisation des Intelligences Artificielles	12
10.1	IA Utilisée : Claude 3.5 Sonnet (Anthropic)	12
10.2	Parties du Code Générées ou Assistées par IA	12
10.3	Parties Réalisées Sans IA	13
10.4	Justification de l'Utilisation	13

11 Conclusion	13
11.1 Objectifs Atteints	14
11.2 Points Forts du Projet	14
11.3 Améliorations Possibles	14
11.4 Compétences Acquisées	14
A Instructions de Compilation et d'Exécution	15
A.1 Prérequis	15
A.2 Compilation	15
A.3 Exécution	15
A.4 Contrôles	15
B Diagramme de Classes (Simplifié)	15
C Extraits de Code Significatifs	15
C.1 Pattern Observer - Notification	15
C.2 Pattern Factory - Création d'Entités	16
C.3 Pathfinding BFS	16

1 Introduction

Ce projet consiste en la réalisation d'un jeu vidéo de type action/aventure inspiré des jeux des années 1990. Le joueur doit échapper à des ennemis et trouver la sortie d'un labyrinthe en collectant des clés pour ouvrir des portes verrouillées.

1.1 Objectifs du Projet

- Concevoir une architecture logicielle solide basée sur le pattern MVC
- Implémenter plusieurs Design Patterns de création, structure et comportement
- Appliquer la théorie des graphes pour le pathfinding des ennemis
- Utiliser l'API graphique Mazing pour l'affichage isométrique
- Respecter les principes du Clean Code

1.2 Scénario du Jeu

Le joueur incarne un chevalier (Crusader) qui doit traverser trois niveaux de labyrinthes progressivement plus difficiles. Des araignées hostiles patrouillent dans le labyrinthe et poursuivent le joueur dès qu'il commence à bouger. Le joueur dispose de 3 vies et doit collecter des clés pour ouvrir les portes verrouillées qui bloquent l'accès à la sortie.

2 Répartition des Responsabilités

Le projet a été réalisé en équipe de trois personnes, chacune avec des responsabilités clairement définies selon le principe de *Separation of Concerns*.

2.1 Membre 1 - Graphes et Pathfinding

Responsabilités :

- Conception et implémentation de la structure de graphe (Node, Edge, MazeGraph)
- Algorithme de Dijkstra pour le calcul du plus court chemin (PathFinder)
- Classes utilitaires pour les positions et directions (Position, Direction)
- Intelligence artificielle des ennemis (pathfinding dans Enemy.update())

Packages : `fr.ubordeaux.ao.project.model.graph`

2.2 Membre 2 - Entités et Factory Pattern

Responsabilités :

- Hiérarchie des entités du jeu (Entity, Player, Enemy, etc.)
- Pattern Factory pour la création d'entités (EntityFactory, EntityType)
- Logique des objets interactifs (clés, portes, sortie)
- Système de vies et de collecte d'objets

Packages : `fr.ubordeaux.ao.project.model.entities, model.patterns.factory`

2.3 Membre 3 - MVC et Interface

Responsabilités :

- Architecture MVC complète du projet
- Pattern Observer pour la communication Model-View (`GameObserver`)
- Vue avec l'API Mazing (`GameView`, `EnemyView`, `HudRenderer`)
- Contrôleurs pour la logique du jeu (`GameController`, `KeyboardController`, `GameLoop`)
- Chargement des niveaux depuis JSON (`Maze`)
- Classe principale et gestion de la progression (`Main`)

Packages : `fr.ubordeaux.ao.project.view`, `controller`

3 Architecture Globale : Le Modèle MVC

Pour garantir une séparation claire des responsabilités, nous avons opté pour une architecture Modèle-Vue-Contrôleur (MVC) stricte. Ce choix répond à la nécessité de découpler la logique du jeu de son interface graphique.

3.1 Le Modèle (`fr.ubordeaux.ao.project.model`)

Le modèle est le cœur de l'application. Il encapsule l'état du jeu (position du joueur, structure du labyrinthe, score) et applique les règles métier.

Principes appliqués :

- **Encapsulation :** Toutes les données sensibles (`lives`, `score`) sont privées. L'accès se fait via des accesseurs (Getters) pour la lecture, et des méthodes métiers (ex : `takeDamage()`) pour l'écriture, garantissant l'intégrité des données.
- **Indépendance :** Le modèle n'a aucune dépendance vers la Vue (aucun import de `view` dans le package `model`). Cela permettrait théoriquement de remplacer l'interface Mazing par une interface console sans changer la logique.

Classes principales :

- `Game` : Classe centrale gérant l'état global du jeu
- `Maze` : Représente le labyrinthe chargé depuis JSON
- `GameConfig` : Configuration centralisée (pattern Singleton partiel)

3.2 La Vue (`fr.ubordeaux.ao.project.view`)

La vue est responsable de la représentation graphique de l'état du modèle. Elle utilise l'API Mazing pour afficher les sprites et Java Swing pour le HUD.

Rôle passif : La vue ne prend aucune décision. Elle se contente de "dessiner" ce que le modèle lui transmet. Elle implémente l'interface `GameObserver` pour être notifiée des changements d'état.

Classes principales :

- `GameView` : Vue principale implémentant `Observer`
- `EnemyView` : Vue des araignées (hérite de `Spider` de Mazing)
- `HudRenderer` : Affichage du HUD (vies, messages)

3.3 Le Contrôleur (fr.ubordeaux.ao.project.controller)

Le contrôleur intercepte les événements utilisateur (clavier) et les transforme en actions sur le modèle.

Boucle de Jeu (Game Loop) : La classe `GameLoop` gère le temps. Elle implémente `Runnable` pour s'exécuter sur un Thread distinct, assurant que la logique du jeu (physique, IA) tourne à une fréquence stable (60 FPS), indépendamment du rendu graphique.

Classes principales :

- `GameController` : Contrôleur principal
- `KeyboardController` : Gestion des entrées clavier (hérite de `KeyAdapter`)
- `GameLoop` : Boucle de jeu principale à 60 FPS
- `EnemyController` : Contrôleur spécifique pour les ennemis

4 Design Patterns Implémentés

Conformément aux exigences du sujet, nous avons intégré quatre patrons de conception pour résoudre des problématiques spécifiques d'extensibilité et de maintenance.

4.1 Pattern Observer (Comportement)

Problème : Comment mettre à jour l'affichage lorsque l'état du jeu change (déplacement, dégâts) sans que le Modèle ne connaisse la classe concrète `GameView` (ce qui violerait le MVC) ?

Solution : Nous avons défini l'interface `GameObserver`.

- Le Modèle (`Game`) agit comme le Sujet. Il maintient une liste d'observateurs.
- La Vue (`GameView`) implémente `GameObserver`.
- Lors d'un changement d'état, le Modèle appelle `notifyObservers()`.

Ce pattern illustre le principe d'inversion de dépendance. Le module de haut niveau (Modèle) ne dépend pas du module de bas niveau (Vue), mais d'une abstraction.

```
1 public interface GameObserver {  
2     void onGameUpdate(Game game);  
3 }
```

Listing 1 – Interface `GameObserver`

```
1 public void notifyObservers() {  
2     for (GameObserver observer : observers) {  
3         observer.onGameUpdate(this);  
4     }  
5 }
```

Listing 2 – Notification dans `Game.java`

4.2 Pattern Factory (Création)

Problème : Le chargement du niveau implique de lire des caractères dans un fichier JSON ('P', 'E', 'D') et d'instancier les objets correspondants (`Player`, `Enemy`, `Door`). Utiliser des `new` directement dans la boucle de chargement créerait un couplage fort.

Solution : La classe `EntityFactory` centralise la création d'objets.

```
1 public static Entity createEntity(EntityType type, Position pos) {
2     switch (type) {
3         case PLAYER: return new Player(pos);
4         case ENEMY_SMALL: return new Enemy(pos, Enemy.Size.SMALL);
5         case KEY: return new Key(pos);
6         case DOOR: return new Door(pos);
7         // ...
8     }
9 }
```

Listing 3 – Méthode Factory

Polymorphisme : La méthode renvoie un type abstrait `Entity`. Le code appelant ne sait pas s'il manipule un `Enemy` ou un `Player`, il sait juste qu'il a une `Entity`. Cela facilite l'ajout de nouvelles entités sans modifier la logique de chargement.

4.3 Pattern Singleton (Configuration)

Usage : La classe `GameConfig` regroupe toutes les constantes du jeu (vitesses, chemins de fichiers, codes couleurs).

Implémentation : Elle utilise des champs `public static final` et un constructeur privé pour empêcher l'instanciation. Bien que ce ne soit pas un Singleton pur au sens "instance unique", cela remplit le même rôle d'accès global unique à la configuration, favorisant le "Clean Code" en évitant les "nombres magiques" dispersés dans le code.

```
1 public class GameConfig {
2     public static final int PLAYER_LIVES = 3;
3     public static final float PLAYER_SPEED = 1.0f;
4     public static final float COLLISION_DISTANCE = 0.8f;
5
6     private GameConfig() {
7         throw new AssertionError("Ne pas instancier");
8     }
9 }
```

Listing 4 – GameConfig.java (extrait)

4.4 Pattern MVC (Architectural)

Le pattern MVC est le pattern architectural principal de notre application. Bien qu'il soit mentionné comme ne comptant pas dans les 3 patterns requis, il structure l'ensemble du projet et garantit la séparation des responsabilités.

5 Algorithmique et Théorie des Graphes

Une partie majeure du développement a porté sur l'intelligence artificielle des ennemis, utilisant la théorie des graphes comme suggéré dans le sujet.

5.1 Modélisation du Labyrinthe

Le labyrinthe est initialement une grille 2D (Matrice). Pour permettre le calcul d'itinéraire, nous l'avons transformé en graphe :

— **Sommets (Nodes) :** Chaque case "marchable" (sol) est un sommet.

- **Arêtes (Edges)** : Une arête relie deux sommets si les cases sont adjacentes physiquement (Nord, Sud, Est, Ouest).
- **Classe MazeGraph** : Cette classe opère la transformation lors du chargement du niveau.

```

1 public void buildGraph(Maze maze) {
2     // Creer un noeud pour chaque case walkable
3     for (int y = 0; y < maze.getHeight(); y++) {
4         for (int x = 0; x < maze.getWidth(); x++) {
5             if (maze.isWalkable(x, y)) {
6                 Node node = new Node(new Position(x, y));
7                 nodes.put(new Position(x, y), node);
8             }
9         }
10    }
11
12    // Creer les aretes entre voisins
13    for (Node node : nodes.values()) {
14        for (Direction dir : Direction.values()) {
15            Position neighborPos = node.getPosition().getNeighbor(dir);
16            Node neighbor = nodes.get(neighborPos);
17            if (neighbor != null) {
18                node.addEdge(new Edge(node, neighbor, 1.0));
19            }
20        }
21    }
22 }

```

Listing 5 – Construction du graphe (MazeGraph.java)

5.2 Algorithme de Recherche de Chemin (Pathfinding)

Pour que les ennemis chassent le joueur intelligemment, nous avons implémenté un algorithme de recherche en largeur (BFS) directement dans la classe **Enemy**.

Fonctionnement : À partir de la position de l'ennemi (source), l'algorithme explore le graphe en cercles concentriques (via une File - Queue) jusqu'à trouver le nœud du joueur (cible).

Reconstruction : Une fois la cible atteinte, nous remontons la chaîne des parents pour obtenir le chemin optimal.

Avantage : Contrairement à une IA aléatoire, cet algorithme garantit le chemin le plus court. L'ennemi contourne les murs efficacement pour piéger le joueur.

```

1 private List<Position> findPath(Position start, Position goal, MazeGraph
2     maze) {
3     Map<Position, Position> parent = new HashMap<>();
4     Queue<Position> queue = new LinkedList<>();
5
6     queue.add(start);
7     parent.put(start, null);
8
9     while (!queue.isEmpty()) {
10        Position p = queue.poll();
11        if (p.equals(goal)) break;
12
13        for (Direction d : maze.getWalkableDirections(p)) {

```



```

13         Position next = p.getNeighbor(d);
14         if (!parent.containsKey(next)) {
15             parent.put(next, p);
16             queue.add(next);
17         }
18     }
19 }
20
21 // Reconstruction du chemin
22 List<Position> path = new LinkedList<>();
23 Position cur = goal;
24 while (cur != null) {
25     path.add(0, cur);
26     cur = parent.get(cur);
27 }
28
29 return path;
30 }

```

Listing 6 – Pathfinding BFS dans Enemy.java

Note : Une classe `PathFinder` implémentant l’algorithme de Dijkstra a également été développée mais n’est pas utilisée dans la version finale. Le BFS intégré dans `Enemy` s’est révélé plus simple et suffisant pour nos besoins.

6 Gestion des Entités du Jeu

6.1 La Classe Enemy

La classe `Enemy` représente les ennemis du jeu (araignées) présents dans les différents niveaux du labyrinthe. Son objectif principal est de créer des entités capables de se déplacer de manière autonome, d’interagir avec le joueur et de respecter les contraintes du labyrinthe.

Attributs clés :

- **Taille** (SMALL, MEDIUM, LARGE) : influence la vitesse de l’ennemi
- **canMove** : indique si l’ennemi peut se déplacer (activé après le premier mouvement du joueur)
- **lastMoveTime** et **moveCooldown** : régulent le rythme des déplacements

La méthode principale `update(Player, MazeGraph, Maze)` effectue les actions suivantes :

1. Vérifie si l’ennemi peut bouger
2. Calcule un chemin vers le joueur à l’aide du BFS
3. Déplace l’araignée d’une case vers le joueur si la case est marchable

Cette logique garantit que les ennemis suivent le joueur de manière autonome tout en respectant les contraintes du labyrinthe.

6.2 Gestion des Clés, Portes et Sorties

Key : La clé représente un objet que le joueur peut ramasser. Lorsqu’il entre en contact :

- La clé est ajoutée à l'inventaire du joueur
- Elle est consommée et retirée du niveau
- Door** : La porte peut être verrouillée ou ouverte :
- Si elle est verrouillée, le joueur peut l'ouvrir s'il possède une clé
- Lorsque le joueur utilise une clé, la porte devient traversable

Exit : La sortie marque la fin d'un niveau et est toujours traversable. L'entité **Exit** ne déclenche pas directement la fin du niveau lors du contact ; cette logique est gérée par le contrôleur du jeu. L'utilisation de `EntityType.EXIT` permet d'identifier facilement la sortie.

7 Qualité du Code (Clean Code)

Nous avons appliqué les principes du "Clean Code" tout au long du développement :

7.1 Nommage Significatif

Les noms de classes et variables sont explicites :

- `isWalkable`, `handlePlayerMove`, `lives`
- Pas de variables cryptiques comme `x1`, `temp`, `data`

7.2 Méthodes Courtes

Chaque méthode réalise une seule tâche (Principe de Responsabilité Unique). Les méthodes longues ont été décomposées en sous-méthodes privées pour améliorer la lisibilité.

7.3 Gestion des Exceptions

Le chargement des fichiers (JSON) est encapsulé dans des blocs `try-catch` pour éviter le crash de l'application en cas de fichier manquant ou corrompu.

```
1 try {
2     BufferedReader reader = new BufferedReader(
3         new InputStreamReader(stream));
4     Gson gson = new Gson();
5     JsonObject json = gson.fromJson(reader, JsonObject.class);
6     // ...
7 } catch (Exception e) {
8     System.err.println("Erreur chargement JSON: " + e.getMessage());
9     e.printStackTrace();
10 }
```

Listing 7 – Gestion des erreurs dans Maze.java

7.4 Documentation

Le code contient des commentaires JavaDoc pour les classes et méthodes principales, facilitant la compréhension et la maintenance.

8 Structure du Projet

8.1 Organisation des Packages

```
fr.ubordeaux.ao.project/  
  Main.java  
  controller/  
    GameController.java  
    GameLoop.java  
    KeyboardController.java  
    EnemyController.java  
  model/  
    Game.java  
    Maze.java  
    GameConfig.java  
    entities/  
      Entity.java  
      Player.java  
      Enemy.java  
      Key.java  
      Door.java  
      Exit.java  
      Wall.java  
    graph/  
      Node.java  
      Edge.java  
      MazeGraph.java  
      PathFinder.java  
      Position.java  
      Direction.java  
    patterns/  
      factory/  
        EntityFactory.java  
        EntityType.java  
      observer/  
        GameObserver.java  
  view/  
    GameView.java  
    EnemyView.java  
    HudRenderer.java  
  test/  
    TestMaze.java  
    TestGameView.java
```

8.2 Ressources

```
src/main/resources/  
  levels/  
    level1.json
```

level2.json

level3.json

9 Fonctionnalités du Jeu

9.1 Niveaux

- **Niveau 1** : Labyrinthe 10×10 , 1 araignée, difficulté facile
- **Niveau 2** : Labyrinthe 15×15 , 3 araignées, difficulté moyenne
- **Niveau 3** : Labyrinthe 20×20 , 4 araignées, difficulté difficile

9.2 Gameplay

- Déplacement du joueur au clavier (ZQSD ou flèches directionnelles)
- 3 vies pour le joueur
- Collecte de clés pour ouvrir les portes verrouillées
- Ennemis inactifs jusqu'au premier mouvement du joueur
- IA des ennemis avec pathfinding intelligent
- Système de score (+1000 points par niveau complété, +100 par ennemi éliminé)
- Progression automatique entre les niveaux
- Game Over et redémarrage automatique
- Écran de victoire finale après le niveau 3

10 Utilisation des Intelligences Artificielles

Conformément aux exigences du professeur, nous déclarons explicitement l'utilisation d'intelligences artificielles dans ce projet :

10.1 IA Utilisée : Claude 3.5 Sonnet (Anthropic)

Version : Claude 3.5 Sonnet via claude.ai
Méthode : Conversation interactive (Chat)
Période : Novembre-Décembre 2024

10.2 Parties du Code Générées ou Assistées par IA

1. Documentation JavaDoc :

- Nous avons utilisé Claude pour générer et compléter les commentaires JavaDoc des classes principales
- Méthode : Copie du code source dans le chat, demande de génération de JavaDoc conforme aux standards
- Fichiers concernés : Toutes les classes des packages `model`, `view`, `controller`

2. Débogage et Correction d'Erreurs :

- Assistance pour identifier et corriger des bugs dans la logique de collision

- Aide pour résoudre les problèmes de synchronisation entre le modèle et la vue
- Méthode : Description du bug observé, analyse du code par l'IA, suggestions de correction

3. Structure du Projet :

- Conseils sur l'organisation des packages selon les principes MVC
- Suggestions d'amélioration de l'architecture
- Méthode : Discussion sur les bonnes pratiques, validation de l'architecture proposée

4. Rapport :

- Génération de la structure du rapport
- Méthode : Fourniture du contenu en français, demande de mise en forme professionnelle

10.3 Parties Réalisées Sans IA

Les éléments suivants ont été entièrement conçus et développés par l'équipe sans assistance d'IA :

- Architecture globale du projet (décisions d'architecture MVC)
- Logique de jeu complète (règles, gameplay, progression)
- Algorithmes de pathfinding (BFS, structure du graphe)
- Pattern Factory et hiérarchie des entités
- Intégration de l'API Mazing
- Fichiers JSON des niveaux
- Logique de la boucle de jeu (GameLoop)
- Tests et validation du gameplay

10.4 Justification de l'Utilisation

L'utilisation de Claude nous a permis de :

- Accélérer la rédaction de la documentation
- Détecter rapidement des erreurs subtiles dans le code
- Améliorer la qualité du rapport final
- Gagner du temps sur les tâches répétitives (documentation)

Important : Tous les membres de l'équipe sont capables d'expliquer et de justifier chaque partie du code, qu'elle ait été assistée par IA ou non. La conception et les décisions architecturales sont entièrement notre travail.

11 Conclusion

Ce projet nous a permis de mettre en pratique les concepts fondamentaux de la programmation orientée objet et de l'architecture logicielle :

11.1 Objectifs Atteints

- Architecture MVC stricte et fonctionnelle
- 4 Design Patterns implémentés (MVC + Observer + Factory + Singleton)
- Théorie des graphes appliquée au pathfinding
- Jeu complet et jouable avec 3 niveaux de difficulté croissante
- Code propre et bien organisé selon les principes du Clean Code
- Séparation claire des responsabilités entre les membres de l'équipe

11.2 Points Forts du Projet

- Architecture solide et extensible permettant l'ajout facile de nouveaux niveaux, ennemis ou objets
- Intelligence artificielle des ennemis efficace grâce au pathfinding
- Gameplay fluide et responsive avec une boucle de jeu à 60 FPS
- Séparation stricte entre la logique métier et l'affichage
- Utilisation judicieuse des Design Patterns pour résoudre des problématiques réelles

11.3 Améliorations Possibles

Si nous devions continuer le développement, nous envisagerions :

- Ajout de nouveaux types d'ennemis avec des comportements différents
- Implémentation d'un système de sauvegarde/chargement de la progression
- Génération procédurale de labyrinthes
- Effets sonores et musique d'ambiance
- Mode multijoueur compétitif ou coopératif
- Système de power-ups et d'objets magiques

11.4 Compétences Acquises

Ce projet nous a permis de développer notre maîtrise de :

- L'architecture logicielle (MVC, séparation des responsabilités)
- Les Design Patterns et leur application pratique
- La théorie des graphes et les algorithmes de pathfinding
- Le travail en équipe avec répartition claire des tâches
- Les principes du Clean Code et de la documentation
- L'utilisation responsable d'outils d'IA pour le développement

Remerciements

Nous remercions l'équipe pédagogique du Master Informatique pour son encadrement, ainsi que les créateurs de l'API Mazing qui nous a permis de nous concentrer sur l'architecture logicielle plutôt que sur les aspects graphiques bas niveau.

A Instructions de Compilation et d'Exécution

A.1 Prérequis

- Java JDK 17 ou supérieur
- Bibliothèque Mazing 2.0.0 (mazing.jar)
- Bibliothèque Gson pour le parsing JSON

A.2 Compilation

```
javac -cp lib/mazing-2.0.0.jar:lib/gson-2.8.9.jar \
      -d bin src/main/java/**/*.java
```

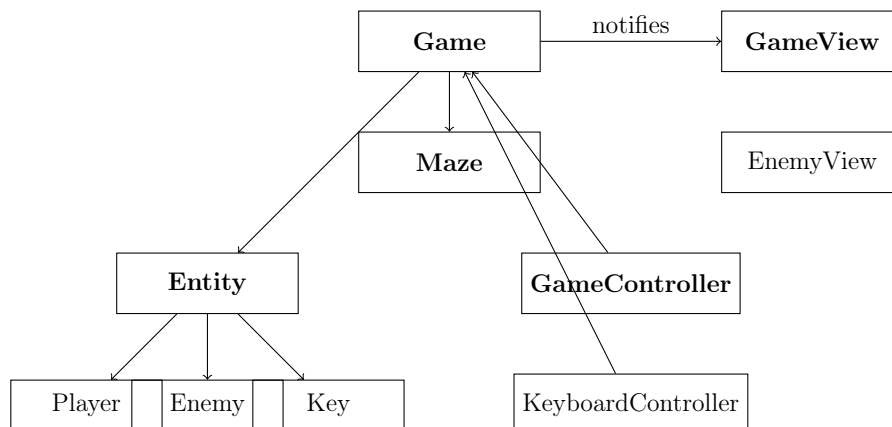
A.3 Exécution

```
java -cp lib/mazing-2.0.0.jar:lib/gson-2.8.9.jar:bin \
      fr.ubordeaux.ao.project.Main
```

A.4 Contrôles

- **Déplacement** : Flèches directionnelles ou ZQSD
- **Objectif** : Collecter les clés, ouvrir les portes, atteindre la sortie
- **Vies** : 3 vies, Game Over si toutes les vies sont perdues

B Diagramme de Classes (Simplifié)



C Extraits de Code Significatifs

C.1 Pattern Observer - Notification

```

1 public void notifyObservers() {
2     for (GameObserver observer : new ArrayList<>(observers)) {
3         observer.onGameUpdate(this);
4     }

```

5 }

Listing 8 – Game.java - Notification des observateurs

C.2 Pattern Factory - Création d'Entités

```

1 public static Entity createEntity(EntityType type, Position pos) {
2     switch (type) {
3         case PLAYER:
4             return new Player(pos);
5         case ENEMY_SMALL:
6             return new Enemy(pos, Enemy.Size.SMALL);
7         case ENEMY_MEDIUM:
8             return new Enemy(pos, Enemy.Size.MEDIUM);
9         case ENEMY_LARGE:
10            return new Enemy(pos, Enemy.Size.LARGE);
11        case KEY:
12            return new Key(pos);
13        case DOOR:
14            return new Door(pos);
15        case EXIT:
16            return new Exit(pos);
17        case WALL:
18            return new Wall(pos);
19        default:
20            return null;
21    }
22 }
```

Listing 9 – EntityFactory.java

C.3 Pathfinding BFS

```

1 private List<Position> findPath(Position start, Position goal,
2                                 MazeGraph maze) {
3     Map<Position, Position> parent = new HashMap<>();
4     Queue<Position> queue = new LinkedList<>();
5
6     queue.add(start);
7     parent.put(start, null);
8
9     // BFS
10    while (!queue.isEmpty()) {
11        Position p = queue.poll();
12        if (p.equals(goal)) break;
13
14        for (Direction d : maze.getWalkableDirections(p)) {
15            Position next = p.getNeighbor(d);
16            if (!parent.containsKey(next)) {
17                parent.put(next, p);
18                queue.add(next);
19            }
20        }
21    }
22
23    // Reconstruction du chemin
```



```
24     List<Position> path = new LinkedList<>();
25     Position cur = goal;
26     while (cur != null) {
27         path.add(0, cur);
28         cur = parent.get(cur);
29     }
30
31     return path;
32 }
```

Listing 10 – Enemy.java - Recherche de chemin